

Pointers and value semantics - Cheat Sheet

Go is pass-by-value. Every function call copies its arguments. That's the rule. The interesting question is what the value contains: the actual data, or a pointer to the actual data.

A pointer holds the memory address of a value. Pass a pointer and the caller can mutate the original. Pass a value and the caller gets a copy. Picking between the two is one of the most consequential design decisions in Go.

Pointer basics

```
x := 42
p := &x      // p is *int, pointing at x
fmt.Println(*p) // 42 (dereference)

*p = 100      // mutate through the pointer
fmt.Println(x) // 100

// Zero value
var q *int    // q is nil
fmt.Println(q) // <nil>
```

Two operators:

- `&` takes the address of a value.
- `*` dereferences a pointer (reads or writes the pointed-to value).

The zero value of any pointer is `nil`. Dereferencing a nil pointer panics.

```
var p *int
*p = 1 // runtime panic: invalid memory address or nil pointer dereference
```

Pointer to struct

Go has no `->` operator. Field access on a pointer works just like field access on a value.

```

type User struct {
    Name string
    Age  int
}

u := User{Name: "alice", Age: 30}
p := &u

p.Name = "bob"           // same as (*p).Name = "bob"
fmt.Println(u.Name) // "bob"

```

`p.Name` is shorthand for `(*p).Name`. The compiler auto-dereferences for field access on pointer values.

new() vs &T

Two ways to allocate a struct and get a pointer:

```

// new returns a pointer to a zero-valued T
p := new(User)
p.Name = "alice"

// Struct literal address-of
q := &User{Name: "alice", Age: 30}

```

`&User{}` is more common because you can set fields at the same time. `new(T)` is rare in practice. The cases where it matters are narrow (allocating a basic type like `new(int)`).

Pointer comparison

Pointers compare with `==` and `!=`. Two pointers are equal if they point at the same address.

```

a := User{Name: "alice"}
b := User{Name: "alice"}

&a == &b    // false: different addresses
&a == &a    // true

```

To compare what the pointers point at, dereference:

```

*p1 == *p2 // compares the values (assuming the value type is comparable)

```

Pointer arithmetic

Not allowed. Go is not C. You can't do `p++` or `p + 1` to move a pointer through memory. If you really need it, `unsafe.Pointer` exists but lives in a separate world.

The lack of pointer arithmetic is a deliberate safety choice. It removes a huge category of memory bugs.

Value vs pointer receivers

When you define a method, the receiver can be a value or a pointer.

```
type Counter struct {
    n int
}

// Value receiver: gets a copy
func (c Counter) Read() int {
    return c.n
}

// Pointer receiver: gets the original
func (c *Counter) Inc() {
    c.n++
}

c := Counter{}
c.Inc()           // works: Go auto-takes the address
fmt.Println(c.n) // 1
```

The compiler can call a pointer-receiver method on a value (`c.Inc()`) when `c` is addressable. It rewrites it to `(&c).Inc()`.

Rules for picking

- **Pointer receiver** when the method mutates the receiver.
- **Pointer receiver** for large structs (avoids copying on every call).
- **Pointer receiver** for any method on a type that already has pointer-receiver methods. Consistency matters.
- **Value receiver** for small immutable types (numbers, small structs, `time.Time`).
- **Value receiver** for types where copying is the intended semantic (`Point`, `Color`, `Vector`).

The single hard rule: keep value and pointer receivers consistent on the same type. `go vet` warns when they're mixed.

Why mixing is bad

The method set of `T` includes value-receiver methods only. The method set of `*T` includes both value and pointer receiver methods. So:

```
type Mutator interface {
    Mutate()
}

type Foo struct{}
func (f *Foo) Mutate() {}

var m Mutator
m = Foo{}    // compile error: Foo doesn't implement Mutator (only *Foo does)
m = &Foo{}  // works
```

This trips up interface implementations constantly.

Pass by value, even for pointers

Every function call in Go copies its arguments. When you pass a pointer, the pointer value is copied (the address). The thing it points to is shared.

```
func reassign(p *int) {
    newVal := 99
    p = &newVal    // local change. Caller doesn't see it.
}

func mutate(p *int) {
    *p = 99        // mutates what p points at. Caller sees it.
}

x := 10
reassign(&x)
fmt.Println(x)    // 10

mutate(&x)
fmt.Println(x)    // 99
```

The same principle applies to slices, maps, and channels. Their headers are passed by value. The underlying data is shared.

Reference-like types

Some Go types behave like references even without explicit pointers.

Type	Behavior
<code>slice</code>	Header is copied (3 words). Backing array is shared.
<code>map</code>	The map value is a pointer to internal state. Mutations are visible to all holders.
<code>channel</code>	Same as map. Internal state is shared.
<code>func</code>	Has a pointer to the function code, may capture state.
<code>interface</code>	Two words (type info + data pointer).
<code>string</code>	2-word header (pointer + length). Immutable, so sharing is safe.

Pointers and these “reference-like” types share a useful property: passing them is cheap and the receiver can read or mutate the shared state (except strings, which are immutable).

For other types (`int` , `struct` , arrays), the whole value gets copied.

```
func f(arr [1000]int) { ... } // copies 8000 bytes per call
func f(s []int)      { ... } // copies 24 bytes (slice header) per call
```

When to use a pointer

- **You need to mutate the caller’s value.**
- **The struct is large** (rule of thumb: more than ~3 fields or any field that’s itself a slice / map / large value).
- **The type already has pointer-receiver methods.** Stay consistent.
- **The type is expensive to copy** (contains a `sync.Mutex` , atomic, file handle, etc.).
- **You want to distinguish “missing” from “zero”.** A `*string` can be nil to mean “not set”, or non-nil pointing at `""` .
- **You’re sharing state across goroutines.** Pointers + sync primitives is the standard model.

When to use a value

- **Small types** (numbers, small structs, time, durations).
- **Types where copying is the right semantic** (`Point` , `Color` , `Vector`).
- **Map keys and comparable constraints** require values that work with `==` .

- **You want zero-value usability.** `var x MyStruct` gives a working zero. `var x *MyStruct` gives a nil pointer.
 - **Types that are immutable by convention.**
-

Optional fields with pointers

Go's zero value gets in the way of “missing vs default” distinctions.

```
type Request struct {  
    Limit int    // 0 could mean "no limit" or "user didn't specify"  
}
```

A pointer disambiguates:

```
type Request struct {  
    Limit *int    // nil = unset, *Limit = user-supplied value  
}  
  
func handle(r Request) {  
    limit := 100 // default  
    if r.Limit != nil {  
        limit = *r.Limit  
    }  
}
```

Cost: every read needs a nil check. Use this pattern only when the distinction matters.

For JSON, `*string` is a common way to distinguish “field absent” from “field equals empty string”. `omitempty` on a pointer drops nil values entirely.

Constructor pattern

Returning a pointer is the standard form for constructors of mutable types.

```
type Server struct {  
    addr    string  
    routes map[string]Handler  
}  
  
func NewServer(addr string) *Server {  
    return &Server{  
        addr:    addr,  
        routes: make(map[string]Handler),  
    }  
}
```

Caller gets a `*Server`. Method calls work the same whether they have value or pointer receivers. The implementation can mutate state without surprising the caller.

For small value types, return a value:

```
func NewPoint(x, y float64) Point {  
    return Point{X: x, Y: y}  
}
```

Functional options

A common Go idiom when constructors need many optional parameters.

```

type Server struct {
    addr      string
    timeout   time.Duration
    logger    *log.Logger
}

type Option func(*Server)

func WithTimeout(d time.Duration) Option {
    return func(s *Server) { s.timeout = d }
}

func WithLogger(l *log.Logger) Option {
    return func(s *Server) { s.logger = l }
}

func NewServer(addr string, opts ...Option) *Server {
    s := &Server{
        addr:      addr,
        timeout:   30 * time.Second,
    }
    for _, opt := range opts {
        opt(s)
    }
    return s
}

// Usage
srv := NewServer("localhost:8080",
    WithTimeout(10*time.Second),
    WithLogger(myLogger),
)

```

The `Option` functions take a pointer because they mutate the struct being built. The constructor returns a pointer for the same reason.

Escape analysis

The compiler decides whether a value lives on the stack or the heap. The stack is per-goroutine and freed automatically when the function returns. The heap is shared and collected by the GC.

A value “escapes to the heap” when its lifetime outlives the function that created it.

```
func stackAlloc() int {
    x := 42    // probably on the stack
    return x
}

func heapAlloc() *int {
    x := 42    // escapes to heap (caller keeps the pointer)
    return &x
}
```

The compiler runs **escape analysis** to figure out which is which. Reasons a value escapes:

- Returning a pointer to it.
- Storing it in a `chan`, `map`, or slice that outlives the function.
- Capturing it in a closure that escapes.
- Passing it as `interface{}` (boxing).

See escape decisions with:

```
go build -gcflags="-m"
```

Stack allocations are much cheaper. Avoiding escapes can speed up hot paths, but profile before chasing it.

Address-of restrictions

You can take the address of most things. A few exceptions exist.

```
type Foo struct {
    N int
}

s := []Foo{{N: 1}}
p := &s[0]    // OK
p.N = 99

m := map[string]Foo{"a": {N: 1}}
p := &m["a"]    // compile error: cannot take the address of m["a"]
```

Map elements aren't addressable because the runtime may move the map's internal storage during a rehash. Take the value, mutate it, put it back:

```
v := m["a"]
v.N = 99
m["a"] = v
```

Or store pointers in the map:

```
m := map[string]*Foo{"a": {N: 1}}
m["a"].N = 99 // works
```

Other non-addressable expressions: function call results, type assertions on values, constant expressions.

nil interface vs interface holding nil pointer

A subtle gotcha that bites every Go dev once.

```
type MyError struct{}
func (e *MyError) Error() string { return "boom" }

func doWork() error {
    var e *MyError = nil
    return e // returns a non-nil error interface
}

err := doWork()
fmt.Println(err == nil) // false. Interface holds (type=*MyError, value=nil).
```

An interface is a 2-word value: (type, data). The interface is nil only if both are nil. Returning a typed nil pointer wraps it in a non-nil interface.

Fix:

```
func doWork() error {
    var e *MyError = nil
    if e != nil {
        return e
    }
    return nil
}
```

Or return `error(nil)` explicitly when nothing went wrong.

Best practices

- **Default to pointer receivers** for most types unless the type is a small value (`Point` , `Color` , `time.Time`).
- **Keep receiver kinds consistent** within a type. All methods value-receiver or all pointer-receiver. `go vet` checks this.

- **Return pointers from constructors of mutable types.** Return values for small immutable types.
 - **Reach for pointers when you need a true optional field** or when the caller must mutate the value.
 - **Avoid taking the address of a loop variable** (especially pre-Go 1.22). The address may be the same across iterations.
 - **Share pointers across goroutines only with synchronization.** Race detector catches this.
 - **Pass slices and maps directly.** They already behave like references. Wrapping them in a pointer (`*[]int`, `*map[string]int`) is almost always wrong.
 - `go vet` and `staticcheck` catch many of these issues. Run them in CI.
-

Common gotchas

- **Nil pointer dereference.** `*p` panics when `p == nil`. Check before deref.
 - **Returning a typed nil pointer through an error interface.** The interface compares non-nil. Return `nil` directly when there's no error.
 - **Address of map element.** Not allowed. Take, modify, put back, or store pointers in the map.
 - **Address of loop variable (pre-Go 1.22).** `for _, v := range s { go func() { use(&v) }() }` captures the same address across iterations. Go 1.22+ gives each iteration its own `v`.
 - **Pointer receiver method called on non-addressable value.** `MyStruct{}.PointerMethod()` is a compile error. The literal isn't addressable. Assign it first.
 - **Mixing value and pointer receivers.** Only `*T` implements interfaces with pointer-receiver methods. `T` values don't satisfy the interface.
 - **Passing a struct with a mutex by value.** Each copy has its own mutex. Locking one doesn't block the other. `go vet` catches the common case.
 - **Copying a `sync.WaitGroup`, `sync.Mutex`, `sync.Once`, `atomic.*`.** They must not be copied. Pass by pointer or embed without copying.
 - **`new(T)` vs `&T{}`.** Almost everyone uses `&T{}`. `new` exists for cases where you need a pointer to a basic type or a zero-valued struct without field assignments.
 - **Escape to heap when you didn't want it.** Capturing a stack variable in a returned closure forces a heap allocation. Use `go build -gcflags="-m"` to see.
-

Quick reference

Operation	Code
Take address	<code>p := &x</code>
Dereference	<code>*p</code>
Pointer type	<code>*T</code>
Allocate zero-valued T	<code>new(T) or &T{}</code>
nil pointer	<code>var p *T</code>
Pointer comparison	<code>p1 == p2</code> (address equality)
Method, value receiver	<code>func (t T) Method()</code>
Method, pointer receiver	<code>func (t *T) Method()</code>
Struct field via pointer	<code>p.Field</code> (auto-deref)
Escape to heap	Returning <code>&x</code> , storing in chan/map, capturing in escaping closure
Not addressable	Map elements, function call results, literals
nil interface	<code>(type=nil, value=nil)</code>
Typed-nil-in-interface	<code>(type=*T, value=nil)</code> compares non-nil
Reference-like types	slice, map, channel, func, interface, string